Problem 1 Garbage Collection

You will complete the implementations of a set of C++ classes which automatically manage the deallocation of MITScript objects using reference counting.

First, we will begin with the reference counting pointer class, named `MySharedPointer`. It has the following properties:

- **Initialization**. `MySharedPointer` can be constructed from a raw pointer to a MITScript object (`Object*`).

```
MySharedPointer shared_ptr1(new IntObject(3));
```

- **Copying**. `MySharedPointer` can be copied from another `MySharedPointer`.

```
MySharedPointer shared_ptr2(shared_ptr1);
```

- **Access**. The object referenced by `MySharedPointer` should be accessed only using the `get` method. Accessing the raw pointer after initialization becomes unsafe.

```
shared_ptr2.get().add(1);
```

- **Deletion**. When all `MySharedPointer`s referring to the same object o all escape lexical scope, o should be automatically deleted using the destructor.

```
void f() {
    Object *o = new IntObject(3);
    MySharedPointer ptr1(o);
    {
        MySharedPointer ptr2(ptr1);
    } // ptr2 deleted here
} //  ptr1 deleted here, o deleted here
```

Below, we provide a simple possible implementation of the `MySharedPointer` class.

```
class MySharedPointer {
private:
    Object *obj;
    int    *ref_cnt;
public:
    MySharedPointer()                                       = delete;
    MySharedPointer &operator=(const MySharedPointer &) = delete;

    MySharedPointer(Object *obj) : obj(obj), ref_cnt(new int(1)) {}

    MySharedPointer(const MySharedPointer &other) :
        obj(other.obj), ref_cnt(other.ref_cnt) {
        (*ref_cnt)++;
    }

    Object &get() {  return *obj; }

    ~MySharedPointer() {
        --(*ref_cnt);
        if (*ref_cnt == 0) {
            delete obj;
            delete ref_cnt;
        }
    }
};
```

With reference counting, cycles can lead to unreachable memory never being reclaimed resulting in memory leaks. One common alternative is to use *weak* reference counting which we implement using a `MyWeakPointer` class.

`MyWeakPointer` has a similar interface to `MySharedPointer`; however, *weak* reference counting will not prevent the referenced object from being deleted. Its implementation should have each of the following properties:

- **Initialization**. `MyWeakPointer` can be initialized from a `MySharedPointer`.

```
MyWeakPointer weak_ptr1(shared_ptr1);
```

- **Copying**. `MyWeakPointer` can be copied from another `MyWeakPointer`.

```
MyWeakPointer weak_ptr2(weak_ptr1);
```

- **Access**. `MyWeakPointer::get` returns a reference to an object if the underlying object has not expired, otherwise it throws an exception.

```
weak_ptr1.get().add(1);
```

- **Deletion**. An object *o* is deleted only when all referencing `MySharedPointer`s escape lexical scope, even if there are `MyWeakPointer`s still referencing *o*.

```
// shared_ptr1, shared_ptr2 go out of scope...
// o is deleted
```

- **Expiry**. `MyWeakPointer::expired` returns true if no `MySharedPointer`s reference the underlying object; otherwise, the method returns false.

```
// shared_ptr1, shared_ptr2 go out of scope...
assert(weak_ptr1.expired() == true);
```

Part a

Rewrite `MySharedPointer` to support *weak reference counting*.

```
class MySharedPointer {
    friend class MyWeakPointer;
private:
    Object *obj;
    int    *strong_ref_cnt;
    int    *weak_ref_cnt;
public:
    MySharedPointer()                                    = delete;

    MySharedPointer &operator=(const MySharedPointer&) = delete;

    MySharedPointer(Object *obj) : obj(obj),
        strong_ref_cnt(new int(1)), weak_ref_cnt(new int(0)) {}

    MySharedPointer(const MySharedPointer &other) :
        obj(other.obj), strong_ref_cnt(other.strong_ref_cnt),
        weak_ref_cnt(other.weak_ref_cnt) {
        (*strong_ref_cnt)++;
    }

    MySharedPointer(const MyWeakPointer &other) :
        obj(other.obj), strong_ref_cnt(other.strong_ref_cnt),
        weak_ref_cnt(other.weak_ref_cnt) {
        (*strong_ref_cnt)++;
    }

    Object &get() {  return *obj; }
    ~MySharedPointer() {
        --(*strong_ref_cnt);
        if (*strong_ref_cnt == 0) {
            delete obj;
            if (*weak_ref_cnt == 0) {
                delete strong_ref_cnt;
                delete weak_ref_cnt;
            }
        }
    }
};
```

Part b

Finish the implementation of `MyWeakPointer`.

```
class MyWeakPointer {
    friend class MySharedPointer;
private:
    Object *obj;
    int    *strong_ref_cnt;
    int    *weak_ref_cnt;
public:
    MyWeakPointer()                                    = delete;
    MyWeakPointer &operator=(const MyWeakPointer &) = delete;
    MyWeakPointer(const MyWeakPointer &other) :
        obj(other.obj), strong_ref_cnt(other.strong_ref_cnt),
        weak_ref_cnt(other.weak_ref_cnt) {
        (*weak_ref_cnt)++;
    }
    MyWeakPointer(const MySharedPointer &other) :
        obj(other.obj), strong_ref_cnt(other.strong_ref_cnt),
        weak_ref_cnt(other.weak_ref_cnt) {
        (*weak_ref_cnt)++;
    }

    bool expired() { return *strong_ref_cnt == 0; }

    Object &get() {
        if (expired())
            throw std::runtime_error("expired");
        return *obj;
    }

    ~MyWeakPointer() {
        --(*weak_ref_cnt);
        if (*strong_ref_cnt == 0 && *weak_ref_cnt == 0) {
            delete strong_ref_cnt;
            delete weak_ref_cnt;
        }
    }
};
```